

UNIVERSIDAD MARIANO GÁLVEZ

SEDE EL NARANJO

Carrera: Ingeniería en Sistemas de información y Ciencias de la computación.

Ingeniero Jose Miguel Villatoro

Curso: Desarrollo Web

Proyecto Final

Manual Técnico

Luis Samuel Menchú Tun 9490 23 6285

Dayna Marianne Meza Maltez 9490-23-3808

Fecha: 27/10/2026

MANUAL TÉCNICO

Control del documento

Documento	Manual técnico de EduMatch
Sistema	Marketplace educativo para conectar estudiantes y tutores
Versión	1.0
Fecha de elaboración	Octubre de 2026

1. Introducción

Este manual técnico describe la organización y los principales componentes de EduMatch, una aplicación web orientada a facilitar la búsqueda de tutores, la publicación de servicios académicos, la consulta de disponibilidad y la coordinación de sesiones entre usuarios. El documento sirve como referencia para comprender la estructura del código, preparar el entorno de desarrollo, ejecutar el sistema y verificar su funcionamiento.

La documentación se apoya en la estructura del proyecto y en las evidencias de código, ejecución y pruebas recopiladas durante el desarrollo. Los nombres de rutas y comandos deben corresponder a los definidos en los archivos de configuración del repositorio.

2. Objetivo y alcance

2.1 Objetivo general

Documentar los elementos técnicos esenciales de EduMatch para facilitar su instalación, mantenimiento, comprensión y validación.

2.2 Alcance

Describir la arquitectura general y la separación entre frontend y backend.

Identificar los módulos funcionales y los modelos de datos principales.

Documentar la autenticación, las rutas de API y los mecanismos de persistencia.

Explicar la ejecución local y la verificación básica del sistema.

Registrar la evidencia de las pruebas automatizadas disponibles.

3. Descripción general del sistema

EduMatch funciona como un marketplace educativo. La aplicación permite gestionar usuarios con diferentes roles, presentar perfiles de tutores, consultar servicios académicos y coordinar sesiones. El backend concentra la lógica de negocio y la validación de solicitudes; el frontend proporciona la interfaz mediante la que interactúan los usuarios.

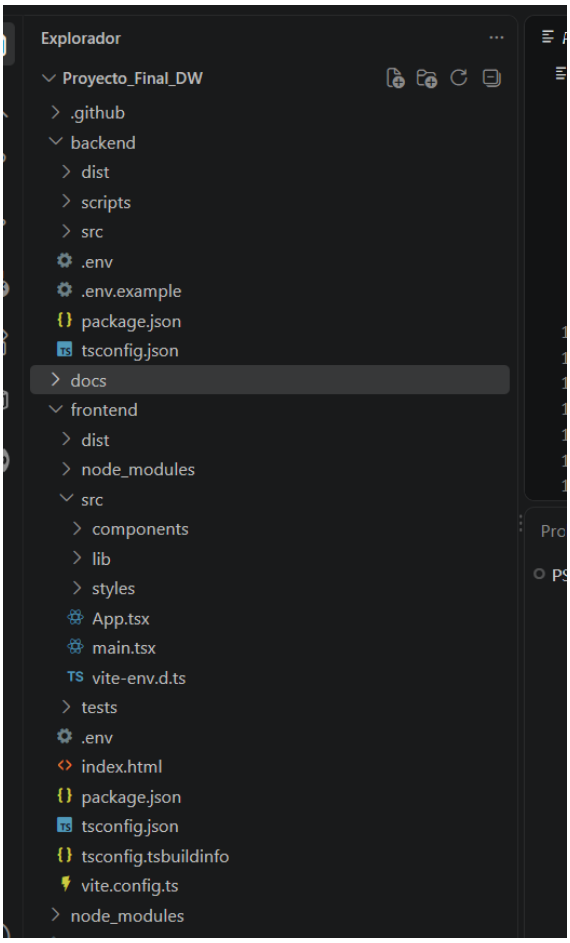
La aplicación utiliza autenticación con Google para identificar al usuario y mecanismos de token para autorizar solicitudes protegidas. La información persistente se gestiona mediante MongoDB y Mongoose. El sistema también incluye componentes para la programación de sesiones, comunicación entre usuarios y funciones administrativas, según los módulos implementados.

4. Arquitectura del sistema

La solución se organiza como una aplicación web con dos componentes principales: frontend y backend. El frontend presenta las vistas y realiza solicitudes a la API. El backend recibe las solicitudes, valida la identidad y los datos, ejecuta las reglas de negocio y consulta o actualiza la base de datos.

4.1 Estructura del repositorio

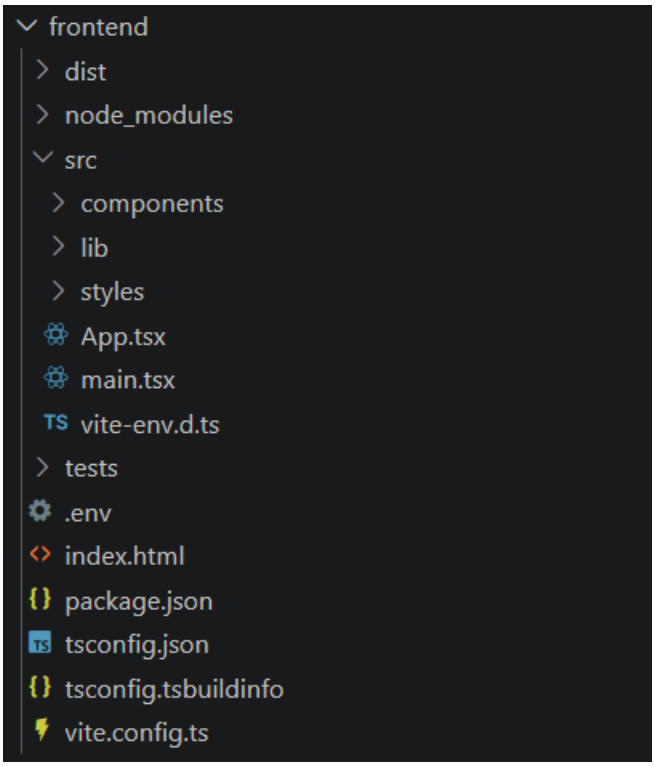
El repositorio contiene una configuración de espacios de trabajo (workspaces) para administrar el proyecto y ejecutar tareas del frontend y del backend desde la raíz. La organización modular permite mantener separadas las responsabilidades de autenticación, usuarios, tutores, servicios, programación y administración.



Estructura general del proyecto en el explorador de archivos.

4.2 Frontend

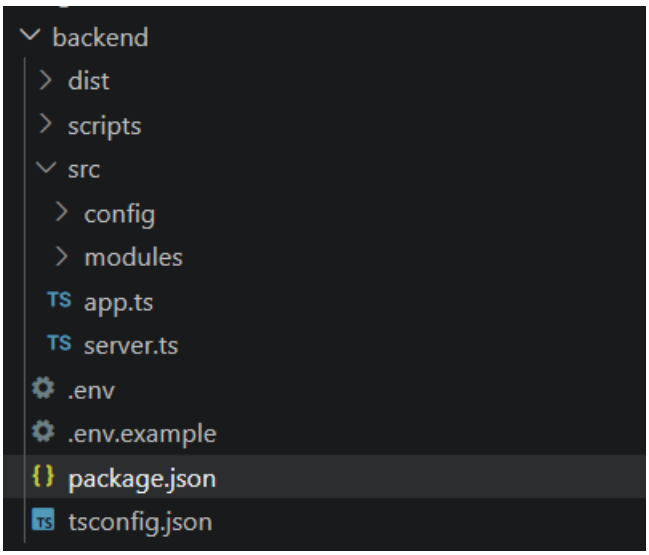
El frontend contiene la interfaz de usuario y la lógica de presentación. Sus responsabilidades incluyen mostrar información, recoger datos de formularios, iniciar acciones solicitadas por el usuario y comunicarse con el backend a través de las rutas de la API. La configuración de desarrollo utiliza Vite cuando así lo establece el proyecto.



Estructura de carpetas y archivos principales del frontend.

4.3 Backend

El backend está organizado en módulos. Esta organización facilita localizar las rutas, controladores, servicios y modelos de cada área funcional. El servidor configura Express, registra las rutas y conecta con los servicios de persistencia y autenticación.



Estructura de carpetas y archivos principales del backend

5. Tecnologías y componentes

Las tecnologías centrales observadas en el proyecto son:

TypeScript y JavaScript para la implementación de la aplicación.

Node.js y npm para la ejecución y administración de dependencias.

Express para la configuración del servidor HTTP y la API.

MongoDB como base de datos documental y Mongoose para definir modelos y validaciones.

Google Identity / verificación de tokens de Google para el proceso de autenticación.

JWT para representar la sesión autenticada en las solicitudes protegidas, según la configuración del backend.

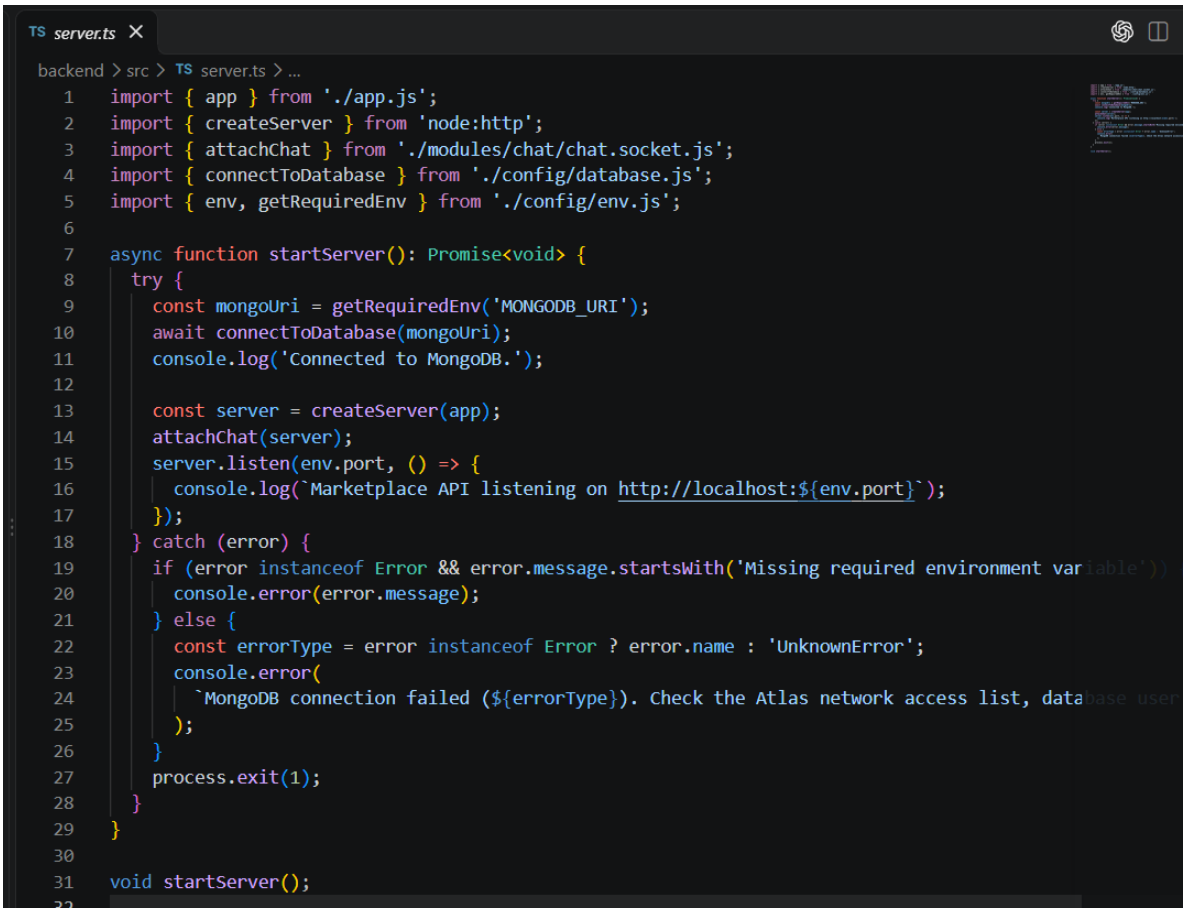
Vite para el servidor de desarrollo del frontend, cuando corresponde a la configuración del proyecto.

Socket.io para funcionalidades de comunicación en tiempo real, si están habilitadas en la configuración de ejecución.

6. Inicialización del servidor y configuración de la API

6.1 Arranque del servidor

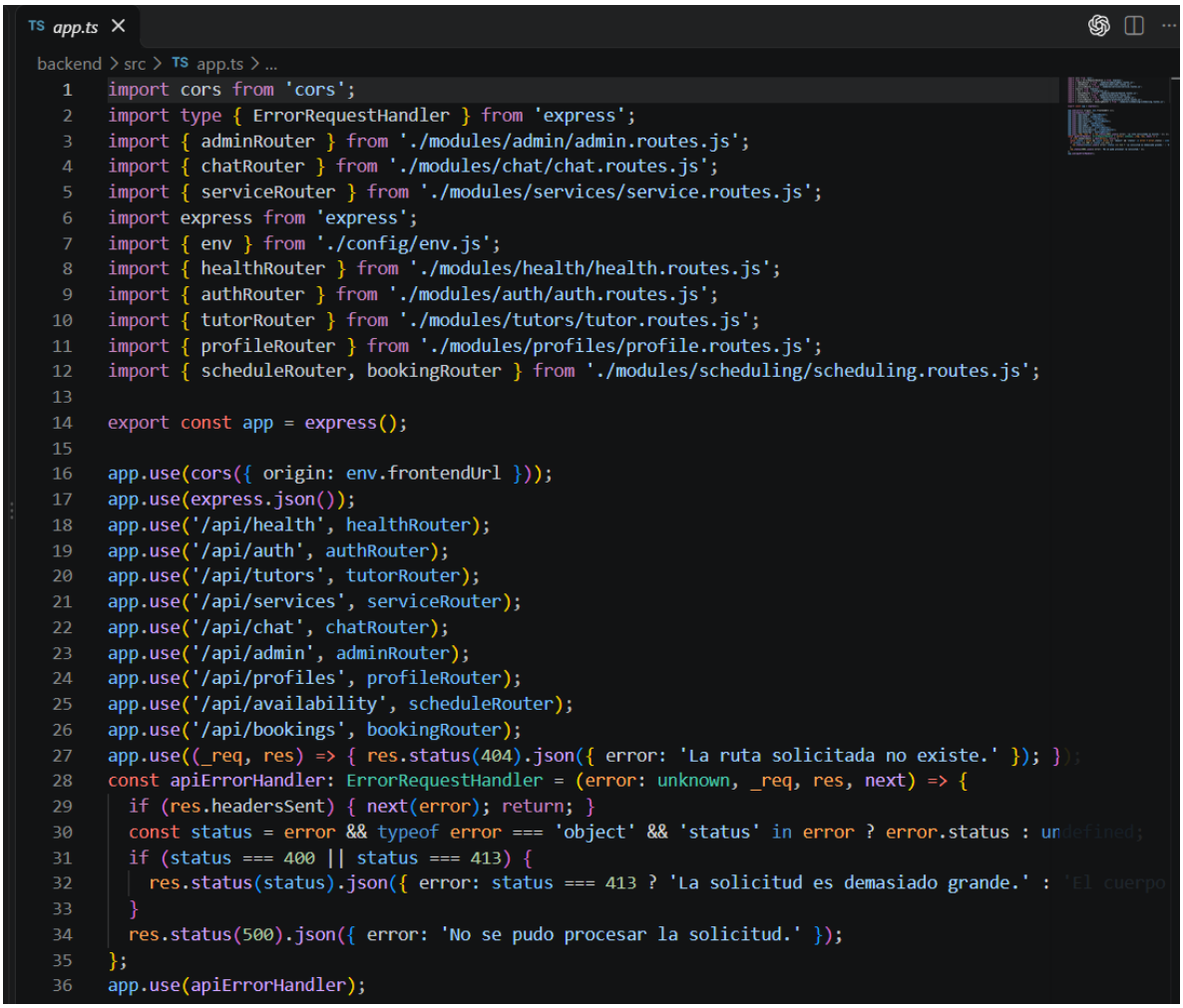
El punto de entrada del backend prepara e inicia el servidor. En esta etapa se cargan las dependencias de configuración, se inicializa la aplicación y se habilita la escucha en el puerto configurado. Las variables sensibles y los parámetros específicos del entorno deben mantenerse en archivos de configuración local o variables de entorno, sin publicarse en el repositorio.



Archivo de entrada del servidor y código de inicialización.

6.2 Configuración de Express y registro de rutas

La configuración de la aplicación Express establece los middlewares generales y registra las rutas de los módulos. Las rutas organizan los endpoints disponibles para que el frontend pueda realizar operaciones de autenticación, consulta y administración.



Configuración de la aplicación y registro de rutas de la API.

6.3 Conexión con MongoDB

La capa de configuración de base de datos establece la conexión entre el backend y MongoDB. Los datos de conexión deben proporcionarse mediante la configuración del entorno. Las credenciales, cadenas de conexión y secretos no deben aparecer en las capturas del manual.

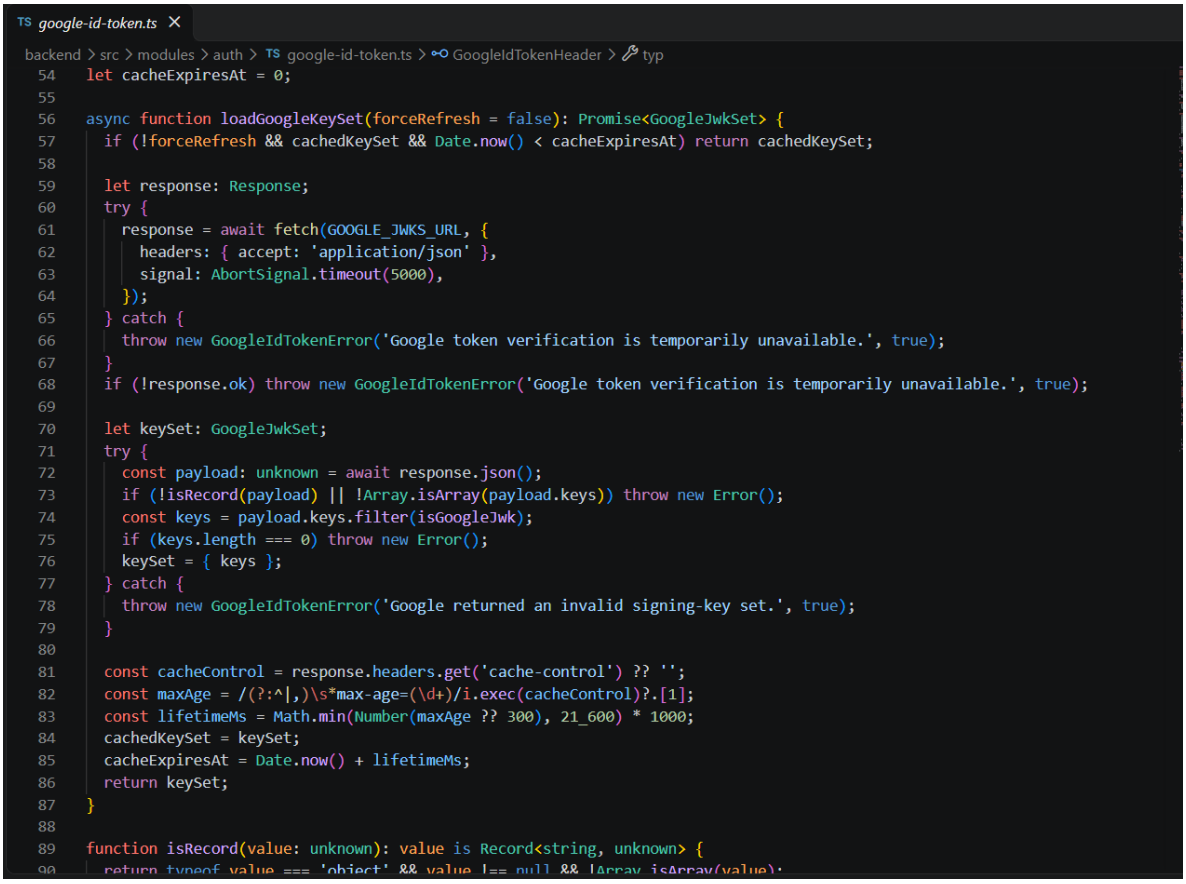


Configuración de conexión con MongoDB; no mostrar secretos.

7. Autenticación y autorización

7.1 Verificación del token de Google

El módulo de autenticación verifica el token de identidad emitido por Google antes de aceptar la identidad del usuario. Entre las comprobaciones implementadas se encuentran el algoritmo de firma permitido, la selección de la clave pública correspondiente, la verificación de la firma y la validación de claims como emisor, audiencia, expiración, identificador del sujeto y correo electrónico verificado.



Código de obtención de claves públicas, verificación del token y validación de claims de Google.

7.2 Tokens de sesión y protección de rutas

Después de identificar al usuario, el backend utiliza los mecanismos de sesión configurados en el proyecto para autorizar solicitudes posteriores. El middleware de autenticación debe comprobar la presencia y validez del token y permitir el acceso únicamente cuando se cumplen las condiciones definidas por la aplicación. Las operaciones administrativas o restringidas deben aplicar los controles de rol correspondientes.



```
export function signAccessToken(
  subject: string,
  secret: string,
  expiresIn: string,
  nowSeconds = Math.floor(Date.now() / 1000),
): string {
  const lifetime = parseDurationSeconds(expiresIn);
  if (!subject || !Number.isSafeInteger(nowSeconds)) throw new Error('Cannot create access token.');

  const payload = {
    sub: subject,
    iss: JWT_ISSUER,
    aud: JWT_AUDIENCE,
    iat: nowSeconds,
    exp: nowSeconds + lifetime,
  };
  const signingInput = `${encodeSegment(JWT_HEADER)}.${encodeSegment(payload)}`;
  return `${signingInput}.${signatureFor(signingInput, secret).toString('base64url')}`;
}
```

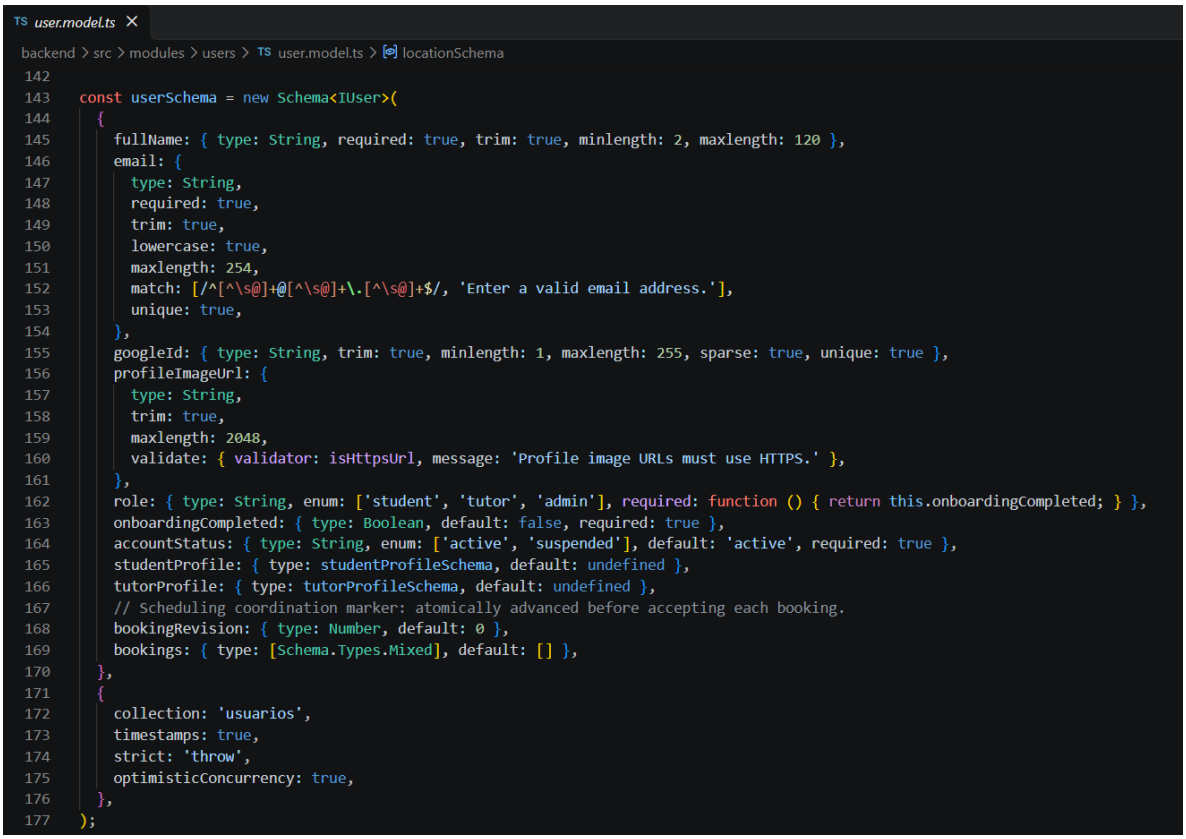
Generación/verificación de JWT o middleware de autenticación, según los archivos disponibles.

8. Modelo de datos

Los modelos de Mongoose definen la estructura de los documentos almacenados en MongoDB, así como restricciones, valores predeterminados y validaciones. La colección configurada para usuarios se denomina “usuarios”.

8.1 Modelo de usuario

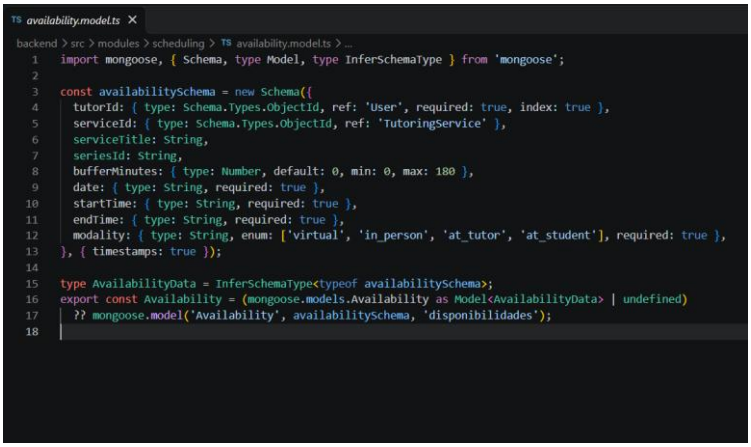
El modelo de usuario contempla datos de identificación, correo electrónico, identificador de Google, rol, estado de la cuenta y datos complementarios para estudiantes y tutores. También incorpora marcas de tiempo y opciones para mantener la consistencia en actualizaciones concurrentes. Los perfiles de estudiante y tutor contienen campos específicos de sus respectivas funciones.



Modelo User, perfiles de estudiante/tutor y validaciones relacionadas.

8.2 Modelos de disponibilidad y reservas

Los modelos relacionados con disponibilidad y reservas representan los horarios habilitados y las sesiones programadas. Sus campos y reglas deben reflejar las condiciones de negocio implementadas, por ejemplo, intervalos válidos, ausencia de conflictos y estados de reserva.



Modelos de disponibilidad y reservas, según la estructura real del módulo.

9. Módulos funcionales del backend

9.1 Servicios académicos

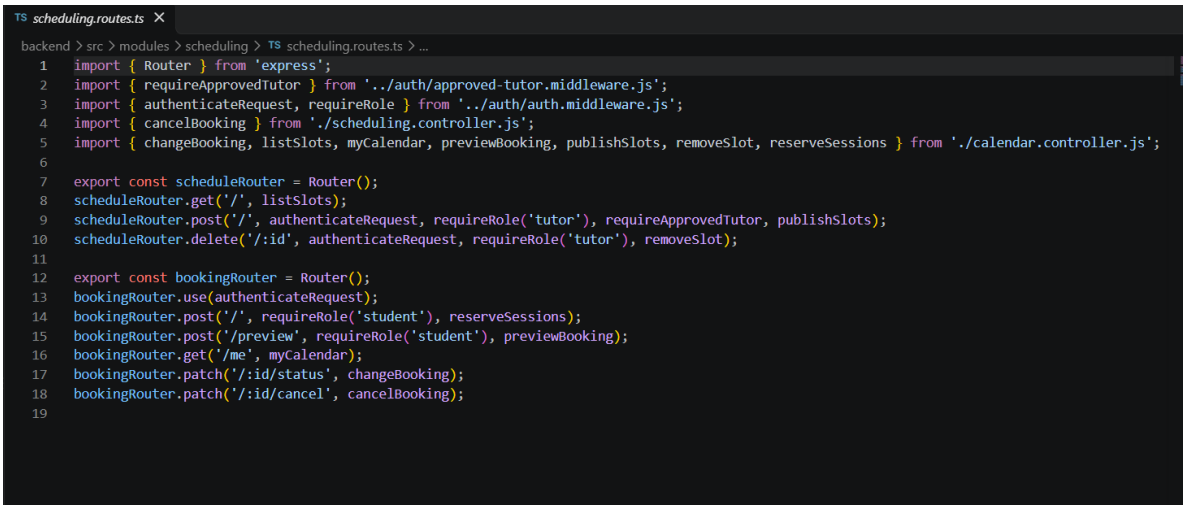
El módulo de servicios académicos gestiona las ofertas de tutoría. Los modelos definen los datos de los servicios; los controladores procesan las solicitudes; y las rutas exponen las operaciones de consulta y gestión mediante la API. Las validaciones deben impedir valores no permitidos y mantener consistentes los datos publicados.



Modelo, controlador o rutas del módulo de servicios académicos.

9.2 Programación de sesiones

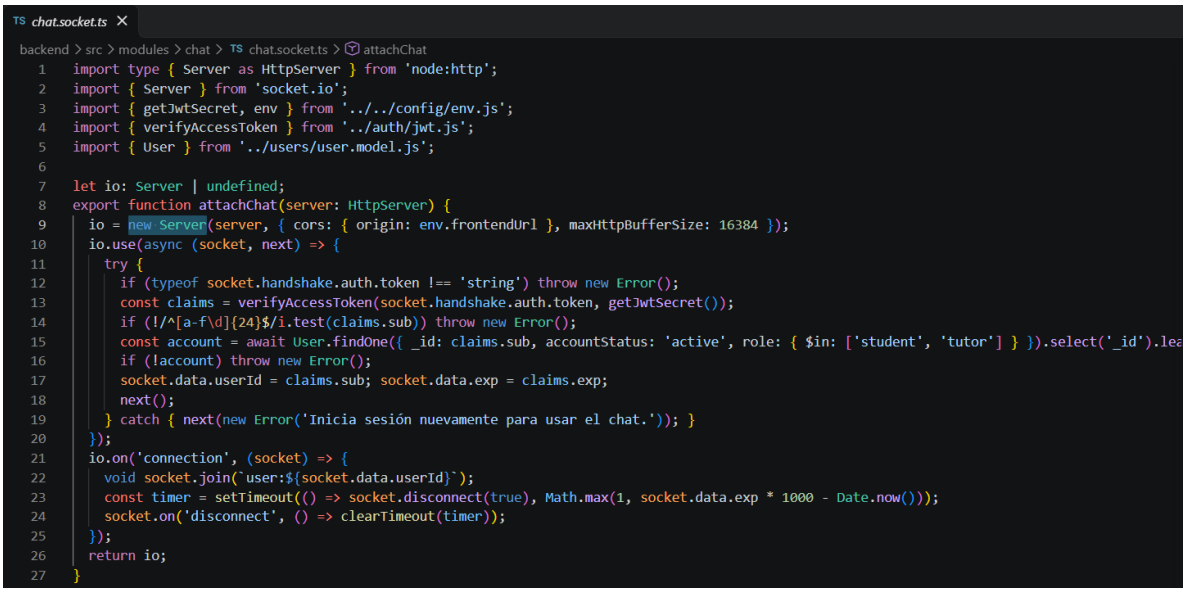
El módulo de programación coordina la disponibilidad y las reservas. La lógica de negocio valida los intervalos horarios y las condiciones necesarias para registrar una sesión. Las pruebas automatizadas incluyen verificaciones relacionadas con intervalos, recurrencias, superposición de sesiones y cálculo de precios, según los casos implementados.



Lógica de agendamiento, validación de disponibilidad o rutas de reservas.

9.3 Chat y comunicación en tiempo real

El componente de chat permite la comunicación entre usuarios. Cuando la funcionalidad en tiempo real está habilitada, Socket.io gestiona las conexiones y los eventos de comunicación; el backend debe validar los participantes y los datos de los mensajes antes de procesarlos.



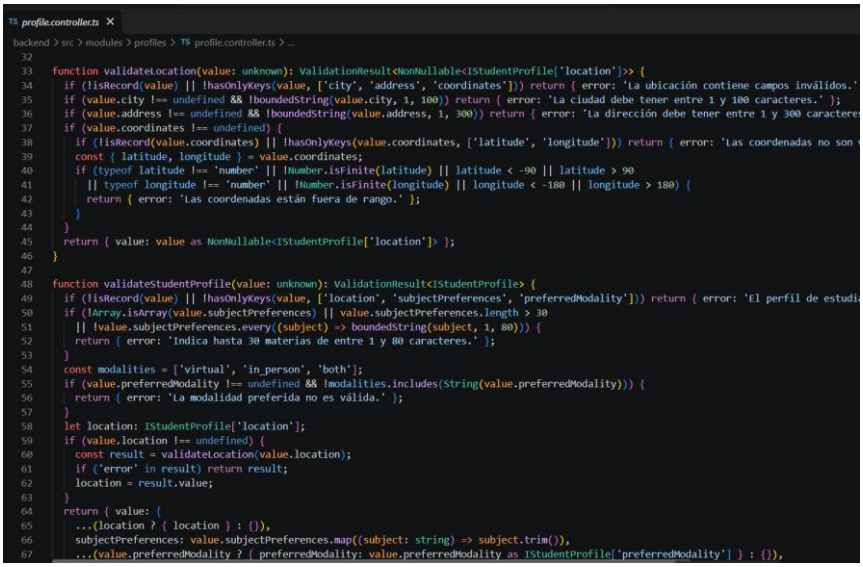
```ts
import type { Server as HttpServer } from 'node:http';
import { Server } from 'socket.io';
import { getJwtSecret, env } from '../../config/env.js';
import { verifyAccessToken } from '../auth/jwt.js';
import { User } from '../users/user.model.js';

let io: Server | undefined;
export function attachChat(server: HttpServer) {
  io = new Server(server, { cors: { origin: env.frontendUrl }, maxHttpBufferSize: 16384 });
  io.use(async (socket, next) => {
    try {
      if (typeof socket.handshake.auth.token !== 'string') throw new Error();
      const claims = verifyAccessToken(socket.handshake.auth.token, getJwtSecret());
      if (!/^[a-f\d]{24}$/i.test(claims.sub)) throw new Error();
      const account = await User.findOne({ _id: claims.sub, accountStatus: 'active', role: { $in: ['student', 'tutor'] } }).select('_id').lea
      if (!account) throw new Error();
      socket.data.userId = claims.sub; socket.data.exp = claims.exp;
      next();
    } catch { next(new Error('Inicia sesión nuevamente para usar el chat.')); }
  });
  io.on('connection', (socket) => {
    void socket.join(`user:${socket.data.userId}`);
    const timer = setTimeout(() => socket.disconnect(true), Math.max(1, socket.data.exp * 1000 - Date.now()));
    socket.on('disconnect', () => clearTimeout(timer));
  });
  return io;
}
```

Configuración de Socket.io o implementación de la gestión de mensajes.

9.4 Administración

El módulo administrativo reúne las operaciones de supervisión y gestión habilitadas en la plataforma. Las rutas administrativas deben estar protegidas para que únicamente los usuarios autorizados puedan ejecutar las operaciones correspondientes.



```ts
function validateLocation(value: unknown): ValidationResult<NonNullable<IStudentProfile['location']>> {
  if (!isRecord(value) || !hasOnlyKeys(value, ['city', 'address', 'coordinates'])) return { error: 'La ubicación contiene campos inválidos.'
  if (value.city !== undefined && !boundedString(value.city, 1, 100)) return { error: 'La ciudad debe tener entre 1 y 100 caracteres.' };
  if (value.address !== undefined && !boundedString(value.address, 1, 300)) return { error: 'La dirección debe tener entre 1 y 300 caracteres
  if (value.coordinates !== undefined) {
    if (!isRecord(value.coordinates) || !hasOnlyKeys(value.coordinates, ['latitude', 'longitude'])) return { error: 'Las coordenadas no son
    const { latitude, longitude } = value.coordinates;
    if (typeof latitude !== 'number' || !Number.isFinite(latitude) || latitude < -90 || latitude > 90
      || typeof longitude !== 'number' || !Number.isFinite(longitude) || longitude < -180 || longitude > 180) {
      return { error: 'Las coordenadas están fuera de rango.' };
    }
  }
  return { value: value as NonNullable<IStudentProfile['location']> };
}

function validateStudentProfile(value: unknown): ValidationResult<IStudentProfile> {
  if (!isRecord(value) || !hasOnlyKeys(value, ['location', 'subjectPreferences', 'preferredModality'])) return { error: 'El perfil de estudi
  if (!Array.isArray(value.subjectPreferences) || value.subjectPreferences.length > 30
    || !value.subjectPreferences.every((subject) => boundedString(subject, 1, 80))) {
    return { error: 'Indica hasta 30 materias de entre 1 y 80 caracteres.' };
  }
  const modalities = ['virtual', 'in_person', 'both'];
  if (value.preferredModality !== undefined && !modalities.includes(String(value.preferredModality))) {
    return { error: 'La modalidad preferida no es válida.' };
  }
  let location: IStudentProfile['location'];
  if (value.location !== undefined) {
    const result = validateLocation(value.location);
    if ('error' in result) return result;
    location = result.value;
  }
  return { value: {
    ...(location ? { location } : {}),
    subjectPreferences: value.subjectPreferences.map((subject: string) => subject.trim()),
    ...(value.preferredModality ? { preferredModality: value.preferredModality as IStudentProfile['preferredModality'] } : {}),
```

Controlador o rutas del módulo administrativo.

10. Instalación y ejecución local

10.1 Requisitos previos

Node.js en una versión compatible con el proyecto.

npm, incluido con Node.js.

Acceso a una instancia de MongoDB configurada para el entorno.

Variables de entorno necesarias para la conexión a la base de datos y los servicios de autenticación.

Visual Studio Code u otro editor compatible con TypeScript y JavaScript.

10.2 Instalación de dependencias

Desde una terminal ubicada en la raíz del repositorio, instalar las dependencias utilizando el procedimiento definido por el proyecto. Si el repositorio utiliza workspaces y dispone de un package-lock.json, normalmente se ejecuta:

npm install

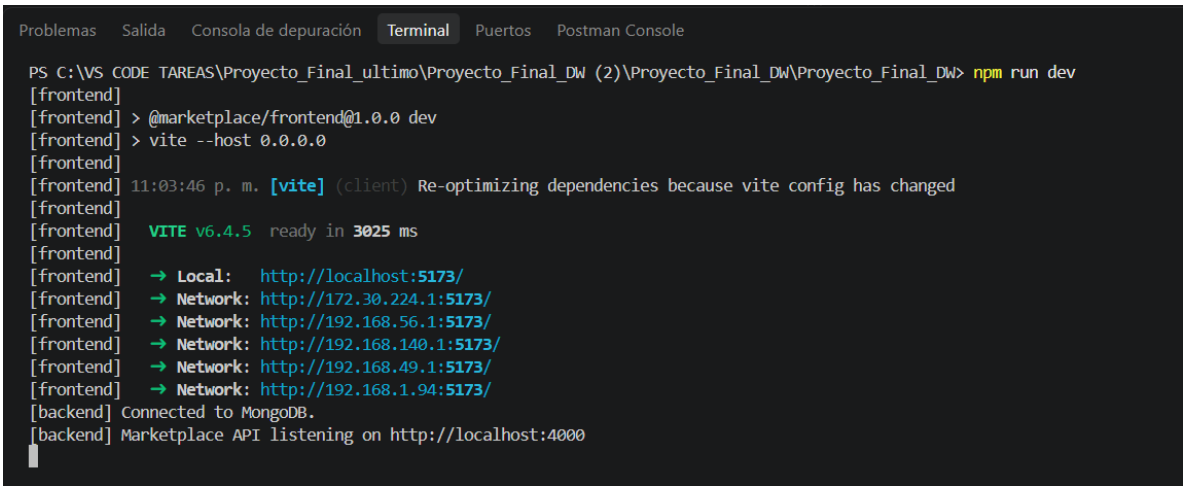
Antes de ejecutar el sistema, comprobar que las variables de entorno están configuradas correctamente y que MongoDB es accesible.

10.3 Inicio del entorno de desarrollo

Ejecutar el script de desarrollo definido en el package.json de la raíz. En el proyecto revisado se utilizó el comando:

npm run dev

Si el script inicia varios procesos, mantener la terminal abierta mientras se utiliza la aplicación. Abrir en el navegador la dirección local que informe Vite o el servidor configurado.

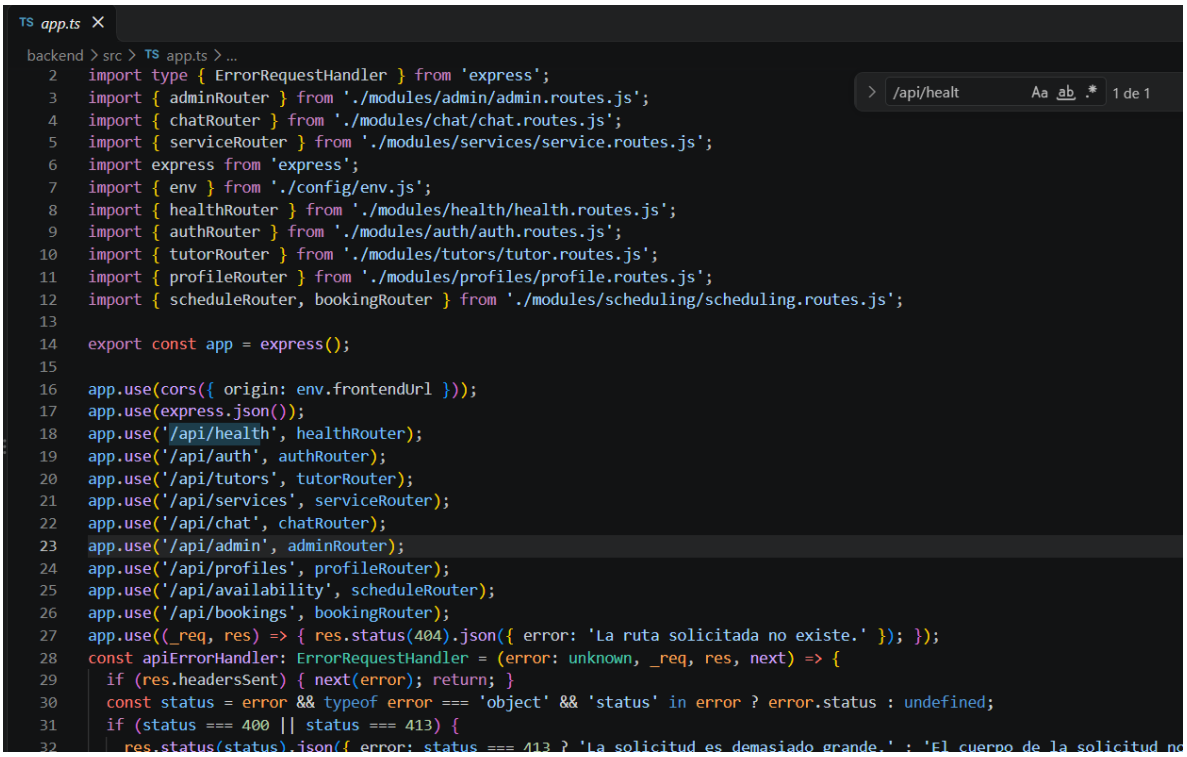


Terminal con los servidores iniciados y/o la interfaz local funcionando.



11. Verificación de disponibilidad de la API

Si el backend implementa el endpoint de salud, este puede utilizarse para verificar que el servidor responde a una solicitud HTTP. Confirmar en el código de rutas la dirección exacta y consultar el puerto real indicado en la configuración. No debe asumirse un puerto o una ruta que no estén definidos en el proyecto.



Respuesta del endpoint de salud de la API, por ejemplo /api/health, si está implementado.

12. Pruebas automatizadas

Las pruebas automatizadas permiten verificar comportamientos concretos del sistema sin depender únicamente de comprobaciones manuales. El script de pruebas de la raíz ejecuta las pruebas configuradas para frontend y backend. En la evidencia recopilada, el frontend reportó 4 pruebas aprobadas y el backend reportó 39 pruebas aprobadas, sin fallos.

Para ejecutar el conjunto de pruebas desde la raíz del repositorio, utilizar:

npm test

Los resultados pueden variar si cambia el código, el entorno o los casos de prueba. Se recomienda ejecutar nuevamente las pruebas después de cambios relevantes y conservar la salida final de la terminal como evidencia.

```
PS C:\VS CODE TAREAS\Proyecto_Final_ultimo\Proyecto_Final_DW (2)\Proyecto_Final_DW\Proyecto_Final_DW> npm test

> marketplace-educativo@1.0.0 test
> npm run test --workspace frontend && npm run test --workspace backend


> @marketplace/frontend@1.0.0 test
> node --import tsx --test tests/*.test.mjs

✓ a late send response cannot undo a read receipt (1.9934ms)
✓ socket retries do not duplicate messages and history stays ordered (20.8411ms)
✓ production chat with a relative API connects to the current website (1.4894ms)
✓ a separate backend uses its origin; development always uses the Vite proxy (0.5062ms)
ℹ tests 4
ℹ suites 0
ℹ pass 4
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 575.9899

> @marketplace/backend@1.0.0 test
> npm run build && node --test dist/modules/auth/auth.test.js dist/modules/profiles/profile.test.js dist/modules/tutors/tutor.test.js dist/modules/tutors/
search.helpers.test.js dist/modules/scheduling/scheduling.test.js dist/modules/scheduling/calendar.test.js dist/modules/services/service.test.js


> @marketplace/backend@1.0.0 build
> tsc -p tsconfig.json
```

```
PS C:\VS CODE TAREAS\Proyecto_Final_ultimo\Proyecto_Final_DW (2)\Proyecto_Final_DW\Proyecto_Final_DW> npm test
✓ single sessions stay single and impossible dates are rejected (2.3943ms)
✓ recurrence rejects invalid weekdays, backwards ranges and excessive series (1.5961ms)
✓ student intervals cannot overlap, but touching sessions are allowed (0.6752ms)
✓ travel buffers block nearby sessions and work across midnight (0.581ms)
✓ prices multiply hours and add one travel charge per session (0.3585ms)
✓ legacy reservations block only valid active intervals (0.7109ms)
✓ availability validation rejects impossible dates and invalid intervals (4.6776ms)
✓ overlap treats touching reservations as separate and rejects intersection (0.4712ms)
✓ optimistic compare-and-swap admits only one concurrent overlapping booking (0.587ms)
✓ services accept virtual lessons without travel charges and home lessons with travel charges (2.1044ms)
✓ services reject invalid amounts and travel charges for virtual or tutor-location lessons (1.0351ms)
✓ services reject identity injection and unsupported categories or modalities (0.4624ms)
✓ search combines course price, modality and literal category (6.7671ms)
✓ presential search includes both locations; home search only home lessons (0.4235ms)
✓ distance calculation handles identical points, known distance and opposite points (0.8276ms)
✓ tutor query builds MongoDB filters for category, price, modality and city (6.0594ms)
✓ tutor query maps virtual modality to virtual profiles (0.5101ms)
✓ exact home modality does not accidentally include tutor-location lessons (0.318ms)
✓ tutor query rejects invalid price ranges and filter values (1.424ms)
✓ public tutor projection omits email, Google id, precise location, and credential evidence URL (0.775ms)
ℹ tests 39
ℹ suites 0
ℹ pass 39
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 1973.7249
PS C:\VS CODE TAREAS\Proyecto_Final_ultimo\Proyecto_Final_DW (2)\Proyecto_Final_DW\Proyecto_Final_DW>
```

Resultados finales de pruebas automatizadas: frontend (4 aprobadas) y backend (39 aprobadas).

13. Seguridad y buenas prácticas

No almacenar claves, contraseñas, tokens privados ni cadenas de conexión en el código fuente o en capturas del manual.

Validar en el backend los datos recibidos desde el frontend; la validación del cliente no sustituye la validación del servidor.

Verificar los tokens de identidad y la autorización antes de permitir operaciones protegidas.

Limitar el acceso a funciones administrativas mediante controles de rol.

Mantener las dependencias actualizadas y revisar avisos de seguridad relevantes.

Registrar errores de forma útil para el diagnóstico, evitando exponer datos sensibles al usuario final.

Utilizar HTTPS en producción y proteger las variables de entorno del despliegue.

14. Diagnóstico de problemas frecuentes

14.1 El servidor no inicia

Revisar los mensajes de la terminal, confirmar que las dependencias estén instaladas y comprobar que el puerto no esté ocupado. Verificar también que el comando utilizado exista en la sección scripts del package.json.

14.2 No hay conexión con MongoDB

Confirmar que la instancia de MongoDB esté disponible y que la variable de conexión corresponda al entorno correcto. No compartir públicamente la cadena de conexión ni sus credenciales.

14.3 La autenticación falla

Comprobar la configuración de las credenciales de Google, la audiencia esperada, la fecha de expiración del token y la configuración de las variables de entorno. Revisar los registros del backend sin publicar tokens ni información privada.

14.4 Las pruebas fallan

Leer el primer error reportado, comprobar si las dependencias están instaladas y ejecutar el comando de pruebas definido por el proyecto. No considerar una ejecución interrumpida como evidencia de éxito.

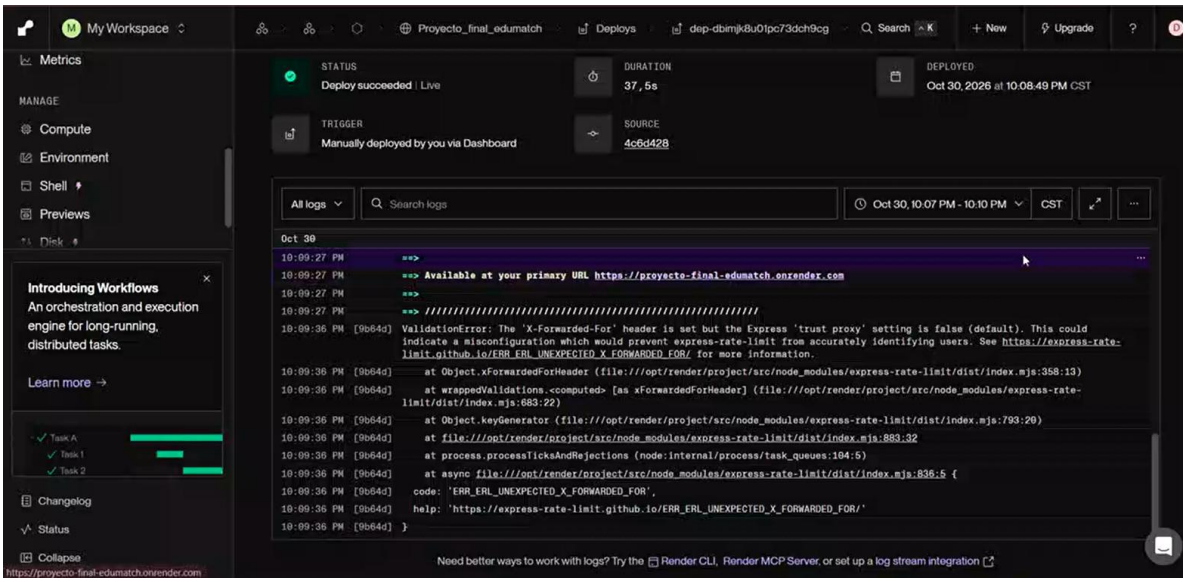
15. Mantenimiento y evolución

Se recomienda mantener una separación clara entre rutas, controladores, lógica de negocio y modelos de datos. Los cambios en el esquema de MongoDB, las reglas de autorización o la programación de sesiones deben acompañarse de pruebas. Antes de desplegar nuevas versiones, ejecutar las pruebas automatizadas y comprobar las variables de entorno del entorno de destino.

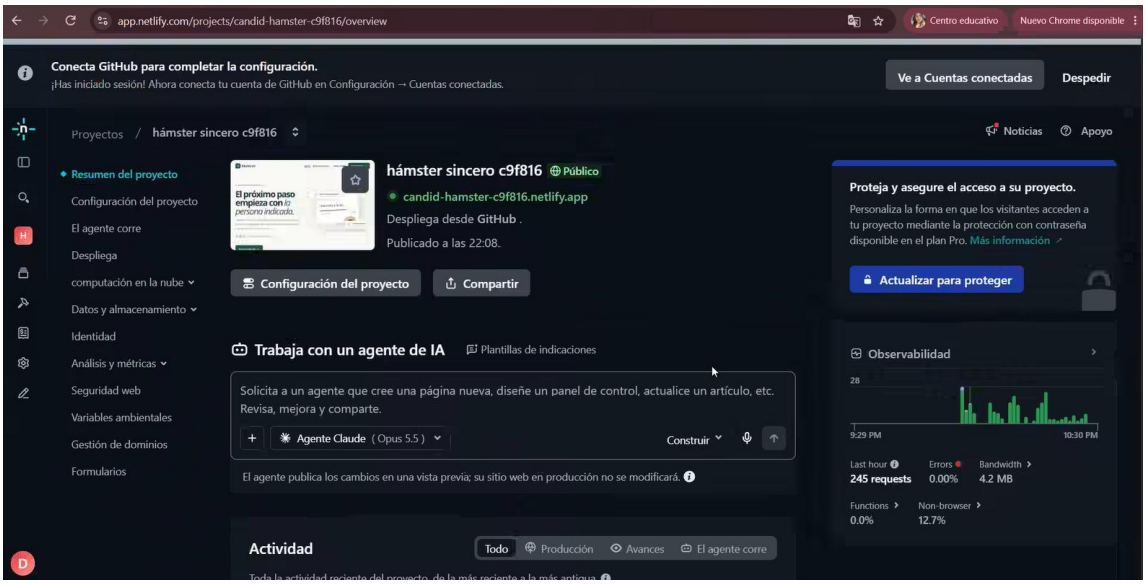
16. Despliegue y servicios externos

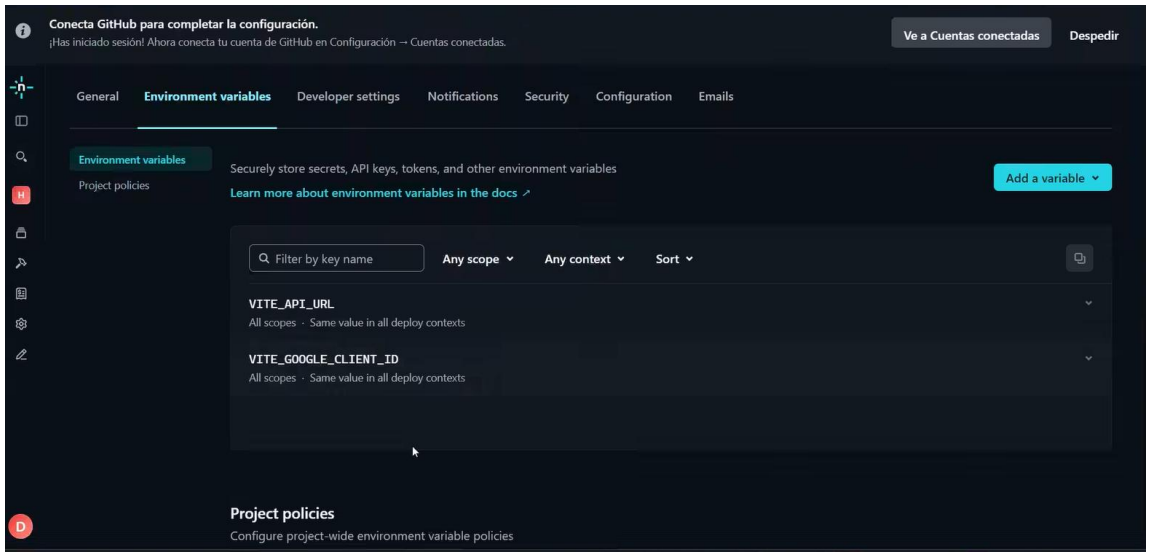
Esta sección reúne evidencias de los servicios externos utilizados para ejecutar, publicar y configurar EduMatch.

16.1 Despliegue del backend en Render

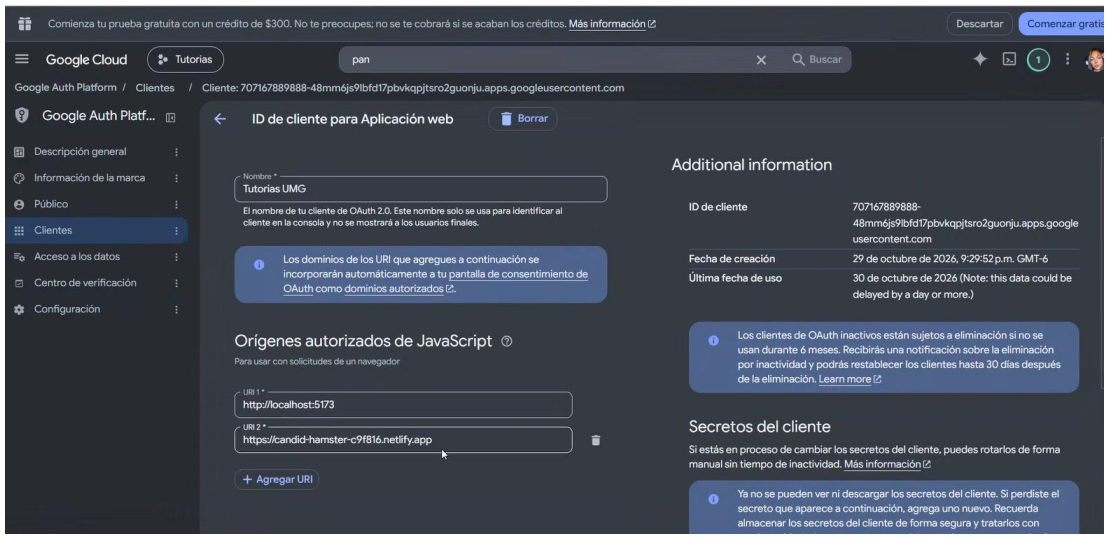


16.2 Publicación del frontend en Netlify

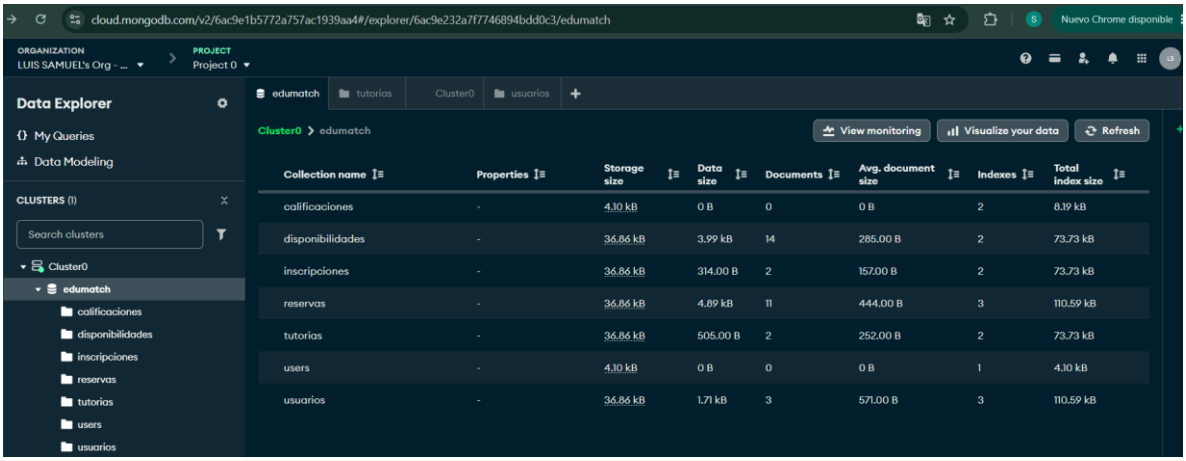




16.3 Configuración de autenticación con Google



16.4 Base de datos MongoDB



17. Conclusión

EduMatch presenta una estructura modular que separa la interfaz web de la lógica del backend y la persistencia de datos. La autenticación, los modelos de Mongoose, los módulos funcionales y las pruebas automatizadas forman parte de los elementos técnicos centrales para su operación y mantenimiento. Este manual proporciona una referencia para comprender la estructura del sistema, ejecutarlo localmente y comprobar su comportamiento, utilizando las evidencias indicadas en cada sección.